CS4104 – COMPUTER GRAPHICS AND VIRTUAL REALITY

PROJECT / CASE STUDY REPORT

TITLE OF THE SELECTED CASE STUDY

CASE STUDY 3: RETAIL AND REMOTE COMMERCE

Submitted by

Nithin S

USN: **1RVU23CSE315**

Under the guidance of

Dr./Prof. **Shyam Mohan J S**

School of Computer Science and Engineering

RV University, Bengaluru

Academic Year: 2026–2027

RV UNIVERSITY

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the CS4104 – Computer Graphics and Virtual Reality case study report titled “Case Study 3: Retail and Remote Commerce” has been prepared and submitted by Nithin S (USN: 1RVU23CSE315) of the School of Computer Science and Engineering, RV University, Bengaluru, during the academic year 2026–2027. The report has been prepared in accordance with the course requirements and instructions provided.

Dr./Prof. Shyam Mohan J S

Faculty Guide

ABSTRACT

This case study explores how computer graphics and virtual reality techniques can support an interior-design startup that wants customers to preview furniture in a home-like setting. The demonstration focuses on a red velvet chair and a brown leather sofa, using three core graphics operations: illumination modelling, texture and normal mapping, and window-to-viewport coordinate conversion.

A desktop showroom built in Unity 6000.3.0f1 is used to make these ideas visible. A custom Phong-style shader separates ambient, diffuse and specular lighting. A procedurally created leather colour map and normal map add surface detail to the sofa while leaving its mesh geometry unchanged. A coordinate example converts the logical point (0.25, −0.30) into the illustrative display position (657.5, 708.0) pixels.

The report explains the mathematics, implementation workflow, user controls, optimization considerations and limitations. Screenshots from an earlier working Unity Play-mode build are included as practical evidence. They confirm that the earlier desktop showroom ran, but they do not verify later presentation changes, measured performance, or deployment on a phone, headset or WebXR platform. The prototype should therefore be understood as an educational desktop demonstration and a foundation for future AR development.

Keywords: Computer graphics, Unity, WebXR concept, Phong illumination, texture mapping, normal mapping, viewport transformation, interactive furniture visualization.

TABLE OF CONTENTS

• 1. Introduction

• 2. Problem Statement and Objectives

• 3. Case Scenario and Requirements

• 4. Relevant CG/VR Concepts

• 5. Technical Analysis and Proposed Solution

• 6. System Design and Workflow

• 7. Performance, Optimization and Limitations

• 8. Applications, Discussion and Practical Evidence

• 9. Conclusion and Future Scope

• References

• Appendices

1. INTRODUCTION

1.1 Background and motivation

Furniture buyers commonly depend on showroom visits and catalogue photographs when deciding what to purchase. Although these methods show a product's general appearance, they may not reveal whether an item will suit a particular room, how its material will react to the room's lighting, or how it will look from different viewing positions. Augmented reality (AR) and virtual reality (VR) can reduce this uncertainty by placing three-dimensional furniture models into an interactive visual environment. To make such previews useful, the rendering, material appearance and screen interaction must work together consistently.

1.2 Relevance of the case study

The selected scenario describes an interior-design startup considering a WebXR-based furniture preview. The virtual red velvet chair should look believable in a bright room, the leather sofa should display fine surface detail without requiring a dense mesh, and user input should correspond to the correct screen region. These requirements connect practical retail visualization with illumination models, surface appearance techniques and coordinate transformations.

1.3 Purpose and scope of the prototype

A compact desktop scene in Unity is used as a controlled learning environment. It contains a red chair, brown sofa, room elements, a window, simple props, adjustable lighting and on-screen controls. Scene objects are generated procedurally during Play mode. The purpose is to demonstrate and compare selected graphics principles; it is not intended to claim photorealistic material reproduction or complete device-based AR functionality. The implementation is associated with the project source repository listed in the references.

2. PROBLEM STATEMENT AND OBJECTIVES

Problemstatement: Develop andexplain a graphics workflowthat keeps furniture visually clear under different lighting conditions, creates convincing material detail without unnecessarily increasing polygon count, and maps logical interaction coordinates accurately to a display viewport, while maintaining a responsive demonstration.

- Demonstrate ambient, diffuse and specular lighting on a simulated red velvet chair.
- Use UV colour texture and normal-map detail to improve the brown sofa's appearance without changing its mesh topology.
- Show a repeatable transformation from logical window coordinates to a phone-style pixel viewport, and relate it to Unity Game-view coordinates.
- Provide a practical Unity workflow with camera interaction, material controls and clear visual evidence.
- Discuss rendering costs, frame-time targets and implementation limitations without presenting unmeasured results as benchmarks.

3. CASE SCENARIO AND REQUIREMENTS

3.1 Scenario and stakeholders

The intended customer experience allows a person to select a chair or sofa, inspect it from several angles, examine its surface characteristics and interact with the model through a display. The startup needs recognizable assets that remain lightweight and behave consistently across screen sizes. Stakeholders include customers comparing products, designers preparing furniture models, graphics developers implementing materials and camera behaviour, and product decision-makers evaluating a future mobile AR or VR release.

3.2 Required behaviour

Requirement	Purpose and proposed approach
Natural chair illumination	Combine an ambient base contribution with Lambertian diffuse response and controlled specular highlights.
Detailed leather surface	Sample UV-based albedo and tangent-space normal maps while retaining the original mesh. Transform a
Accurate screen coordinates	point from a logical coordinate interval into a destination pixel rectangle.
User exploration	Support camera orbit, zoom and reset, object focus and selectable graphics modes.
Readable demonstration	Arrange the room and interface for a practical landscape display.

3.3 Functional and non-functional requirements

Functional features include furniture rendering, selectable lighting modes, separate texture and bump switches, coordinate sliders, an inset-view option and keyboard/mouse input. The non-functional goals are clear presentation, predictable operation, low dependency overhead, reasonable rendering cost and reproducibility in the specified Unity version. A production WebXR application would also need device tracking, object placement, occlusion, camera permissions, performance profiling and accessibility testing; these are beyond the current desktop prototype.

3.4 Validation boundaries

The supplied evidence includes screenshots from an earlier working Unity Play-mode build. They show the rendered room and its graphical panels. Later source-level presentation updates were made, but the earlier images cannot be treated as proof that those newer changes were executed. Similarly, AR interactions described in this report are proposed extensions rather than implemented device-tracking features.

4. RELEVANT COMPUTER GRAPHICS AND VR CONCEPTS

4.1 Rendering pipeline and illumination

A 3D mesh passes through a sequence of operations before appearing on screen. Its vertices are transformed from local space into world and camera space, projected, clipped and rasterized. During rasterization, attributes such as surface normals and UV coordinates are interpolated across triangles. Fragment shading then uses these values to calculate the final surface colour. The Unity example uses a compact forward-shading approach in ShaderLab/HLSL so that the lighting contributions can be inspected directly.

4.2 Ambient, diffuse and specular reflection

Ambient lighting supplies a basic level of illumination, including on surfaces that receive little direct light. Diffuse lighting describes how a matte surface responds to the angle between its normal vector N and the light direction L. The Lambertian factor is max(N·L, 0), preventing negative contributions on surfaces facing away from the light. Specular reflection produces highlights according to the relationship between the viewer direction V and the ideal reflected direction R. The exponent n controls how concentrated the highlight appears.

$$I = k_a I_a + \sum [\, k_d \max(N \cdot L, 0) + k_s \max(R \cdot V, 0)^n \,]$$

For the chair, a strong red base colour and restrained specular response suggest a fabric-like finish. The sofa uses a brown textured base and a more noticeable highlight to imply leather. These settings are illustrative and are not measurements of real furniture. A production AR renderer would need environment-light estimation, consistent shadows, tone mapping and colour matching with the camera image.

4.3 Phong shading compared with PBR

Phong shading is useful for teaching because its ambient, diffuse and specular components are easy to identify. Physically based rendering (PBR) uses parameters such as roughness and metallic response, with a stronger emphasis on energy-consistent material behaviour. PBR is generally a better direction for production consistency, while the simpler Phong model is retained here because it directly demonstrates the concepts required by the case study.

4.4 Texture mapping and UV coordinates

A sofa mesh defines the shape of the object, but it does not automatically contain the fine grain associated with leather. Texture mapping places a two-dimensional image over the mesh by assigning UV coordinates to its vertices. The rasterizer interpolates these coordinates across each visible triangle, allowing the fragment shader to sample the appropriate colour from the albedo map.

$$\mathbf{Albedo(u,v) = FurnitureTint \times TextureSample(AlbedoTexture, u, v)}$$

The texture modifies the visible colour pattern while leaving the object's vertex positions and polygon structure intact.

4.5 Bump mapping and normal mapping

Bump mapping simulates small changes in surface orientation, while normal mapping stores those directions more directly. A tangent-space normal map stores x, y and z components in RGB channels. The shader converts the stored values from the [0,1] range into signed vector components, then uses a tangent-bitangent-normal (TBN) basis to transform the vector into world space. The resulting perturbed normal affects diffuse and specular shading, creating the appearance of pores and creases without changing the mesh boundaries.

$$\mathbf{n_{tangent} = normalize(2 \times RGB_{normal\ map}(u,v) - 1)}$$
$$\mathbf{n_{world} = normalize(T \times n_x + B \times n_y + N \times n_z)}$$

4.6 Trade-offs and limitations

Normal maps provide an economical way to suggest fine detail, but they cannot alter the silhouette, create genuinely displaced surface features or correct poor UV layouts. They also require texture memory and additional fragment-shader sampling. In this project, the colour and normal textures are generated procedurally at runtime using deterministic calculations. The updated source specifies two 256 × 256 RGBA textures and separate controls for enabling or disabling the effects. The visual result still depends on texture sampling, lighting, camera angle and normal strength.

4.7 Distinguishing the techniques

Texture mapping changes the surface's colour pattern. Bump or normal mapping changes the normals used for shading. Tessellation or additional mesh geometry changes the actual shape. These methods serve different purposes. The sofa demonstration keeps its mesh fixed and allows colour texture and bump effects to be switched independently, directly addressing the requirement to improve leather detail without increasing polygon count.

4.8 Window-to-viewport transformation

A graphics window defines the logical coordinate region in which a point is specified. A viewport defines the destination rectangle where that point is displayed. The transformation normalizes the point within the source interval and scales it into the destination interval. This is important when matching pointer, touch or projected-marker positions to a phone or headset display.

) × (x

$$\mathbf{x_v = x_{vmin} + (x_w - x_{wmin})(x_{vmax} - x_{vmin})/(x_{wmax} - x_{wmin})}$$
$$\mathbf{y_v = y_{vmin} + (y_w - y_{wmin}) \times (y_{vmax} - y_{vmin})/(y_{wmax} - y_{wmin})}$$

The source intervals must have non-zero widths. Graphics calculations often use a bottom-left origin, whereas many interface systems place their origin at the top-left. In such interfaces, the vertical coordinate may need to be inverted. Screen pixels, normalized viewport coordinates and 3D world positions must not be mixed.

4.9 Connection to 3D interaction

When a virtual chair is rendered on a phone, its points are transformed by model, view and projection matrices before viewport mapping. Picking an object from touch input follows the reverse idea: the screen position is converted into a ray or world-space query using camera and projection information. The prototype demonstrates two-dimensional viewport mapping and camera-rectangle changes, but it does not implement AR plane detection or headset gaze selection.

4.10 Colour representation

The chair and sofa use RGB colours and textures. The final displayed colour is influenced by display transfer functions and the rendering colour space. A production renderer may perform lighting in linear space and treat image textures as sRGB data to improve colour compositing. The teaching shader clarifies lighting contributions, but it is not a calibrated system for reproducing the exact colour of real fabric. Consequently, the same red chair may appear different on different displays.

5. TECHNICAL ANALYSIS AND PROPOSED SOLUTION

5.1 Question 1 – Illumination and colour models

The red velvet chair must remain recognizable while responding naturally to the simulated room lighting. The proposed solution combines a stable ambient contribution, directional diffuse lighting and controlled specular highlights.

5.1.1 Ambient contribution

Ambient illumination prevents areas outside direct light from becoming completely black. It can be represented as the surface albedo multiplied by an ambient-strength value. A modest level helps the chair remain readable in the showroom. In a real AR system, this value should be estimated from the surrounding environment rather than chosen arbitrarily.

5.1.2 Diffuse contribution

A fabric surface appears brighter when it faces the window or simulated light source and darker when it turns away. The term max(N·L, 0) represents this directional effect. Since velvet is generally more matte than polished leather, diffuse lighting contributes strongly to the chair's perceived volume. The custom shader calculates this term using per-fragment normals and the directional-light colour.

5.1.3 Specular contribution

A specular highlight becomes stronger when the viewer looks from a direction close to the ideal reflection direction. The dot product max(R·V, 0) is raised to a shininess exponent to control the highlight's sharpness. The simplified red fabric uses a lower specular coefficient, while the sofa receives a more visible highlight. Keyboard options 1, 2 and 3 allow users to compare ambient-only, ambient-plus-diffuse and complete Phong lighting modes.

5.1.4 Implementation and evaluation

The Unity controller constructs the room, assigns the custom shader to the furniture and sets up a directional light and simulated ambient environment. Moving the camera changes the view direction and therefore changes the specular response. The demonstration is intended to make the three lighting components understandable. A real AR implementation would also need device-based light estimation, shadow information, exposure handling, occlusion, colour calibration and low interaction latency.

5.2 Question 2 – Texture mapping and bump mapping

The leather sofa can be made more detailed without adding polygons by retaining a relatively simple mesh and changing the material information sampled during fragment shading.

5.2.1 Albedo texture

Each mesh vertex carries a UV coordinate. These coordinates are interpolated across visible triangles and used to sample the brown leather albedo map. The sampled pattern introduces colour variation associated with leather grain without adding vertices. In the Unity project, a deterministic pattern and colour modulation generate a 256 × 256 texture at runtime. The interface and T shortcut switch the colour texture on or off.

5.2.2 Normal texture

The procedural pattern is numerically differentiated across neighbouring texture pixels to create a normal texture. The fragment shader decodes the RGB value into a signed vector, scales its tangent-space x and y components using a configurable bump-strength value, and transforms it through the local tangent basis. The resulting normal is used for diffuse and specular calculations. The B key or the material control independently toggles this effect, while the mesh topology remains unchanged.

5.2.3 Efficiency and limitations

Normal mapping represents fine leather irregularities using texture samples rather than extra triangles and geometry processing. This shifts some cost to fragment-shader calculations and texture access, which can be useful on mobile GPUs with limited resources. However, large folds, holes and silhouette changes still require real geometry. Normal mapping does not physically move the surface. More accurate production materials may also require PBR shading and textures captured from real materials.

5.2.4 Evidence and observation

The earlier Play-mode screenshot shows the sofa in the showroom and displays the texture and bump controls. This confirms that the demonstration interface was present, but it does not quantitatively prove the difference between the two toggle states. The source includes independent _UseTexture and _UseBump shader parameters. If a direct visual comparison is required, matching before-and-after screenshots should be captured with the same camera position and lighting.

5.3 Question 3 – Mapping interaction coordinates to the viewport

To map a user's logical interaction point to a phone or VR display, first normalize the point relative to the source rectangle, then scale and translate it into the destination pixel rectangle.

5.3.1 Example values

Parameter	Value used
Logical x and y interval	−1 to +1 on both axes
Selected logical point	(xw, yw) = (0.25, −0.30)
Pixel viewport bounds	x = 70..1010; y = 120..1800
Coordinate origin	Bottom-left for the mathematical example

5.3.2 Worked calculation

$$x_v = 70 + [(0.25 - (-1))/(1 - (-1))] \times (1010 - 70)$$
$$= 70 + (1.25/2) \times 940 = 657.5 \text{ pixels}$$

$$y_v = 120 + [(-0.30 - (-1))/(1 - (-1))] \times (1800 - 120)$$
$$= 120 + (0.70/2) \times 1680 = 708.0 \text{ pixels}$$

Mapped pixel location: (657.5, 708.0)

5.3.3 Unity viewport and interface behaviour

The “Window to Viewport” panel displays the mapped phone-style coordinate and allows the logical x and y values to be adjusted. Unity’s camera viewport is a different rectangle, represented by Camera.rect and Camera.pixelRect. An Inset/Fill option modifies the camera’s rendered area. Since Unity GUI elements commonly use a top-left origin, the marker’s vertical position must be flipped when drawing a bottom-left-origin mathematical coordinate. A headset uses separate eye viewports and projection matrices, but the same normalization principle applies.

6. SYSTEM DESIGN AND WORKFLOW

6.1 Layered design

The application separates scene construction, user interaction, material shading and screen-coordinate conversion. This separation makes the demonstration easier to understand and troubleshoot. For example, camera movement can be changed without modifying the sofa mesh or its texture, and the viewport marker can be adjusted without changing the lighting shader.

Conceptual processing flow: Scene setup → Furniture and material creation → Lighting and camera setup → User input → Vertex/fragment processing → Game viewport and interface output

6.2 Rendering sequence

When the application starts, the controller creates primitive furniture geometry, applies the custom shader, generates procedural leather textures, adds a directional light and configures the camera. During each frame, input handling updates the selected lighting mode, focused object, camera orbit and material switches. Vertex processing transforms the mesh, while fragment processing evaluates the texture samples, perturbed normals and lighting terms. The camera then displays the rendered scene in the active Game viewport, and the interface presents the related controls.

6.3 Main Unity components

Project component	Role
CIE3_Retail_Furniture_Visualizer.unity	Saved launch scene containing controller references and shader setup.
RetailARCaseStudyDemo.cs	Creates furniture, textures, lighting, camera, controls and viewport mapping. Calculates
PhongFurniture.shader	per-fragment ambient, diffuse and specular lighting with albedo and normal-map support.
CIE3SceneBuilder.cs	Provides an Editor menu for opening the saved scene and optionally preparing the visualizer.
CIE3Validation.cs	Contains source-level or Editor checks for the scene, shader and mapping calculations.

6.4 Steps to run the demonstration

- Open the project with Unity 6000.3.0f1 and the Built-in Render Pipeline.
- In Unity Hub, open the CASE STUDY CIE3/Unity_Retail_AR_Demo folder containing Assets, Packages and ProjectSettings.
- Open Assets/Scenes/CIE3_Retail_Furniture_Visualizer.unity, or use the CGVR CIE3 menu item.
- Press Play and select the Game view to generate the room and furniture.
- Use R to reset the camera and keys 1, 2 and 3 to compare the Phong lighting stages.
- Choose Sofa and press F to focus it; use T for the albedo texture and B for the bump effect.
- Press V to open the mapping panel and restore the logical point that maps to (657.5, 708.0).

6.5 Controls and interface

Camera orbit changes the viewing angle, the scroll wheel adjusts zoom, and the left/right arrow keys rotate the selected furniture. Sliders expose lighting and mapping parameters. The newer source also contains presentation and camera-framing improvements, but the screenshots included here come from the earlier build. The prototype does not access a phone camera, create AR anchors or support tracked VR controllers.

7. PERFORMANCE, OPTIMIZATION AND LIMITATIONS

7.1 Rendering constraints

A desktop educational scene and a mobile WebXR application operate under different performance limits. Smooth interaction depends on CPU work, GPU shading and timely frame presentation. VR headsets may render separate views for each eye and require particularly low motion-to-photon latency. The following frame budgets are theoretical design targets, not measured results from this project.

Target frame rate	Time available per frame
60 frames/second	1000 / 60 = 16.67 ms 1000
72 frames/second	/ 72 = 13.89 ms 1000 / 90 =
90 frames/second	11.11 ms

7.2 Optimization choices

• Simple primitive geometry keeps the scene manageable while allowing material textures to provide much of the surface detail.

• Albedo and normal maps avoid increasing sofa tessellation solely to show small pores.

• The custom forward shader exposes its coefficients clearly, although fragment operations and texture samples still consume GPU resources.

• A single controlled lighting setup makes comparisons easier to repeat than a scene with many changing real-time lights.

• Appropriate camera framing and well-positioned interface controls improve usability without adding complex geometry.

7.3 Memory estimate and measurement integrity

Two 256 × 256 RGBA32 texture bases require 2 × 256 × 256 × 4 = 524,288 bytes, or 512 KiB, before accounting for mipmaps, padding or runtime driver overhead. This is a calculation based on the source texture dimensions, not a profiler measurement. Actual performance also depends on model complexity, draw calls, shader variants and the target device.

7.4 Suggested validation

The Unity Profiler and Stats window can be used to record frame time and draw calls under repeatable conditions. Texture and bump settings should be compared using fixed camera positions. Before a phone or headset release, the project should be tested on the target hardware and WebXR runtime, including texture compression, occlusion and input latency. This report does not claim a measured frame-rate benchmark or successful execution on mobile or headset hardware.

8. APPLICATIONS, DISCUSSION AND PRACTICAL EVIDENCE

8.1 Relevance to retail visualization

The demonstration illustrates that a useful furniture preview requires more than a basic 3D model. The chair's lighting response and the sofa's material detail help communicate product appearance, while screen-coordinate mapping supports taps and on-screen hotspots. In a real room, environment-aware lighting would further help virtual furniture blend with the physical surroundings.

8.2 Value of the educational prototype

The Unity project is relatively small, shareable as source code and does not depend on external artwork. Its exposed controls make graphics concepts visible instead of hiding them inside a high-level material preset. Students can compare Phong lighting stages, inspect albedo and normal mapping separately, and verify a known viewport coordinate. This makes the application suitable for a short demonstration and for explaining the mathematical ideas behind the three case-study questions.

8.3 Integration challenges

A real AR furniture application would need access to a camera feed or XR compositor, calibration of physical scale, reliable plane detection, occlusion with room objects, consistent shadow estimation and a touch-to-ray interaction process. Furniture models would also need verified dimensions, suitable UV layouts and dependable textures. Mobile browsers introduce further limits involving GPU capacity, memory, device temperature, permissions and screen variation. The current project does not claim to address these deployment requirements.

8.4 Ethical and practical accuracy

A furniture preview could mislead a customer if it exaggerates material texture, colour or product dimensions. A reliable retail tool should communicate that lighting and material appearance may be approximate and that results depend on the device. The procedural leather and simplified chair in this project are teaching examples, not scientifically measured reproductions of commercial fabrics. The report therefore distinguishes implemented code, observed Play-mode evidence and proposed future capabilities.

8.5 Practical Unity evidence – full room

The first practical image is an original student-provided screenshot captured in Unity 6000.3.0f1 Play mode on 18 October 2026. It shows the earlier working interface, before later layout and presentation changes were committed.

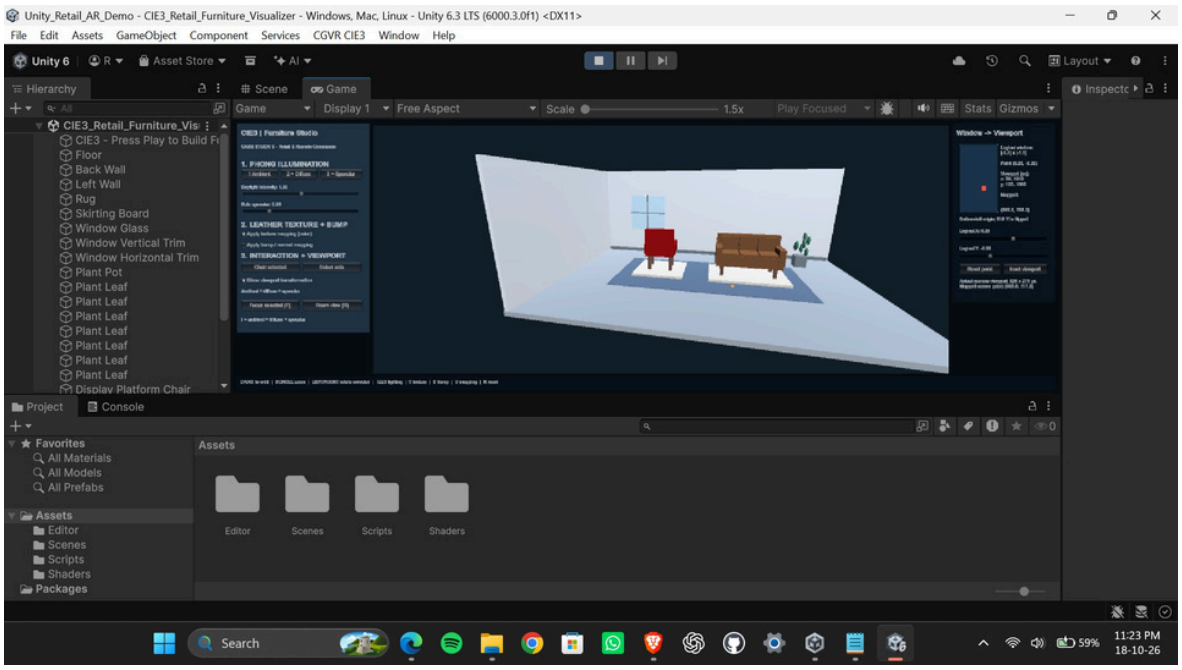


Figure 1. Unity Editor Game view from the earlier working build, showing the red chair, brown sofa, simulated room and controls for lighting, material mapping and viewport transformation.

The wide showroom view contains both furniture objects, a window, floor, display platforms and separate interface areas. It demonstrates that the earlier visualizer rendered its 3D environment in Play mode. It does not establish a measured performance result or confirm that every keyboard control works in the latest source.

8.6 Practical Unity evidence – close view

The second screenshot uses a closer camera position. It shows the chair, sofa material and the illumination/transformation panel more prominently.

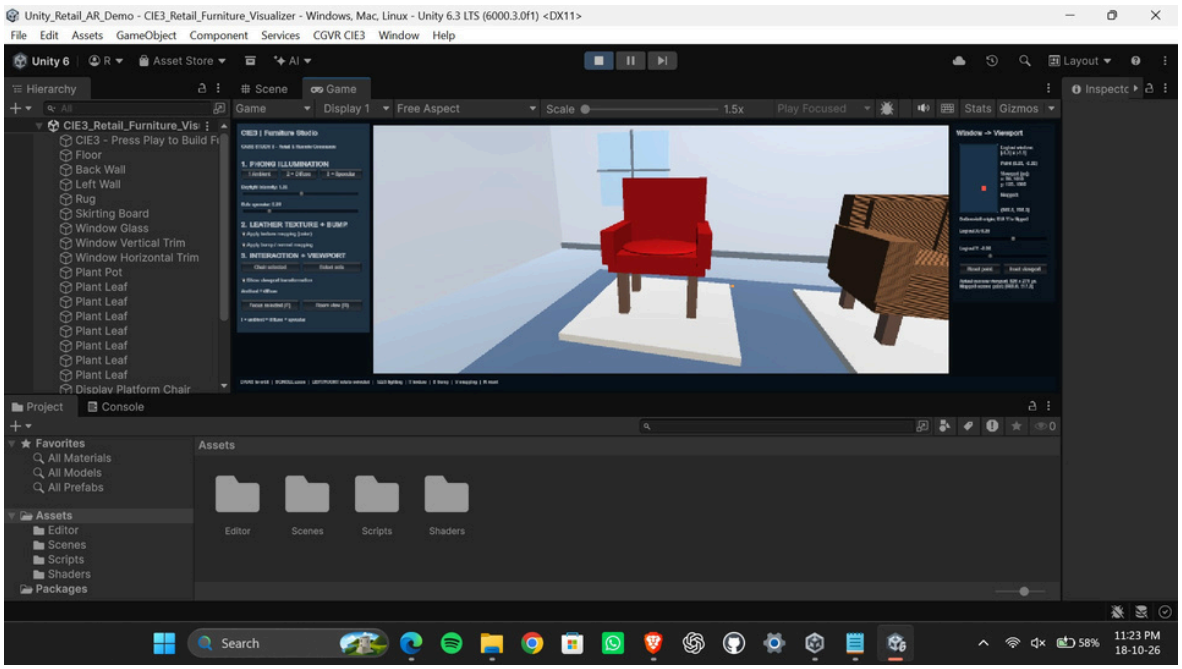


Figure 2. Close Unity Editor Play-mode view from the earlier build, showing the chair material, part of the leather sofa and the mapping panel.

The closer view demonstrates camera movement, but the older layout partly crops the sofa and makes some interface text difficult to read in a non-maximized Game tab. This motivated later source changes to camera framing, panel text size and presentation mode. New screenshots are required before those improvements can be described as visually validated.

8.7 Evidence status

Supported by screenshots: an earlier Unity Play-mode scene containing two furniture objects and a demonstration interface. **Supported by source inspection:** procedural material toggles, coordinate equations and later code changes. **Not independently verified by the included evidence:** the latest build's runtime appearance, separate A/B lighting or bump captures, benchmark FPS, physical smartphone AR, headset operation or a deployed WebXR experience. No result is presented as tested if it was not run.

9. CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

This case study applies core computer graphics concepts to the RV University Retail and Remote Commerce scenario. The red velvet chair demonstrates how ambient, diffuse and specular terms determine a surface's response to lighting direction and viewer position. The brown sofa shows how UV colour textures and per-pixel normal changes can suggest detailed leather without increasing the mesh's polygon count. The viewport example converts the logical point (0.25, −0.30) into the display coordinate (657.5, 708.0) pixels.

These ideas are combined in a Unity showroom containing procedurally created objects, a custom shader, interactive controls and a reproducible scene. The supplied screenshots establish that an earlier version ran in Unity

Play mode. The main strength of the work is its clear connection between code, equations and visible output. Its limitations include simplified materials, the absence of measured performance data and the lack of live AR/VR device tracking. It is therefore a working desktop teaching demonstration of rendering fundamentals, rather than a deployed WebXR commerce application.

9.2 Future enhancements

• Adopt PBR materials, realistic furniture meshes and measured or properly aligned textures to improve visual fidelity.

• Estimate room lighting from camera frames and introduce contact shadows, occlusion and accurate physical scale.

• Add device-specific screen picking, tracked AR placement and stereo VR views with suitable interaction controls. • Measure CPU/GPU frame times and memory on representative mobile devices, then optimize geometry and texture compression.

• Capture comparable full-screen images for each graphics mode and validate the updated presentation build locally.

REFERENCES

[1] RV University, “CS4104 – Computer Graphics and Virtual Reality: Project / Case Study Assignment,” School of Computer Science and Engineering, faculty-supplied course document, 2026.

[2] M. Rishikesh, “Computer-Graphics-and-Virtual-Reality: CASE STUDY CIE3/Unity_Retail_AR_Demo,” GitHub source repository. Available: https://github.com/Rishikeshsanin/Computer-Graphics-and-Virtual-Reality

[3] Unity Technologies, “Unity Manual and Scripting API,” documentation for Camera.rect, Camera.pixelRect, textures and ShaderLab. Available: https://docs.unity3d.com/

[4] B. T. Phong, “Illumination for computer generated pictures,” Communications of the ACM, vol. 18, no. 6, pp. 311–317, Jun. 1975.

[5] T. Akenine-Möller, E. Haines, and N. Hoffman, Real-Time Rendering, 4th ed., CRC Press, 2018.

[6] J. F. Blinn, “Simulation of wrinkled surfaces,” SIGGRAPH Computer Graphics, vol. 12, no. 3, pp. 286–292, 1978.

APPENDIX A – SOURCE AND EXECUTION DETAILS

Unity version: 6000.3.0f1. Rendering path: Built-in Render Pipeline. Programming technologies: C# and ShaderLab/HLSL. The CASE STUDY CIE3/Unity_Retail_AR_Demo folder contains Assets, Packages and ProjectSettings. The saved scene is CIE3_Retail_Furniture_Visualizer.unity. The main controller is RetailARCaseStudyDemo.cs, and the shader is PhongFurniture.shader.

APPENDIX B – FACULTY CHECKLIST

• One case study selected: Case Study 3 – Retail and Remote Commerce.

• All three scenario-specific questions are addressed in Section 5.

• The coordinate transformation is derived with substituted values in Section 5.3.

• System workflow, tables and captioned practical figures are included.

• Frame-time targets, limitations and optimization decisions are discussed.

• The report retains a cover and unsigned certificate format, with numbered references in IEEE style.